

Cyberpunk Velocity

Integrantes

Iván Sardón Medina
José Miguel Valdivia
Rodrigo Silva Murillo

Junio de 2026

Contents

1	Resumen	2
2	Objetivo y alcance	2
3	Organizacion del codigo	2
4	Librerias propias y externas	2
5	Funciones clave aplicadas	3
6	Flujo de ejecucion	4
7	Transformaciones y camaras	4
8	Carga de modelos y texturas	5
9	Iluminacion, materiales y niebla	6
10	Ciudad y Porsche	6
11	Cielo, animaciones y controles	7
12	Trazabilidad visual y tecnica	8
13	Aspectos tecnicos	8
14	Conclusion	9

1 Resumen

Cyberpunk Velocity es una escena 3D interactiva desarrollada con OpenGL 3.3 Core Profile. El programa muestra un Porsche en una carretera urbana nocturna con edificios, vegetacion, postes de luz, semaforo, cielo sintetico, sol con halo, niebla y varias camaras. El archivo `main.cpp` coordina la inicializacion de GLFW/GLAD, la carga de modelos y texturas, la configuracion de proyeccion/vista, la actualizacion temporal de la animacion, el registro de luces y el dibujo final de cada frame.

La escena se apoya en cabeceras propias que funcionan como un pequeno motor grafico: `draw2D.h`, `draw3D.h`, `lighting.h`, `transformations.h`, `camera.h`, `CameraSystem.h`, `objetos.h`, `Porsche.h` y `Sky.h`. Estas librerias encapsulan matrices, shaders, VAO/VBO/EBO, carga OBJ, texturas, materiales, luces, grupos de formas, camaras y objetos compuestos.

2 Objetivo y alcance

El objetivo tecnico es integrar los temas principales de graficos por computadora en una escena navegable: transformaciones 3D, proyeccion perspectiva, camaras `lookAt`, carga de mallas OBJ, texturizado, iluminacion Blinn-Phong, blending, niebla y animacion por tiempo.

El alcance implementado incluye una ventana 1920x1080, un Porsche dividido en piezas, un bloque urbano con carretera y elementos decorativos, un cielo procedural texturizado, luces direccionales y spotlights, materiales con texturas PNG, transparencias, rotacion de ruedas, desplazamiento de capas para simular avance y frenado automatico ante el semaforo.

3 Organizacion del codigo

La estructura logica del programa es:

```
Cyberpunk Velocity/
  main.cpp           // loop principal e inicializacion
  draw2D.h / draw3D.h // base de dibujo, VAO/VBO, formas y OBJ
  lighting.h         // luces, materiales y shader Blinn-Phong
  transformations.h  // Mat4, Transform y composicion TRS
  camera.h           // lookAt, perspective y camara libre
  CameraSystem.h     // modos de camara e input
  objetos.h          // ciudad, carretera, postes, semaforo
  Porsche.h          // vehiculo, piezas, texturas y ruedas
  Sky.h              // domo de cielo, sol y halo
  Models/            // OBJ, MTL y texturas PNG
```

Aunque existen archivos `.mtl`, el cargador propio no interpreta materiales MTL. Las propiedades visuales se asignan manualmente desde C++ mediante `Material`, `Texture2D` y `enableLighting(...)`. Esto da control directo sobre color, especularidad, emisividad, alfa y mapas difusos.

4 Librerias propias y externas

Archivo	Uso principal
<code>transformations.h</code>	Define matrices column-major compatibles con OpenGL. Sus funciones clave son <code>Mat4::identity</code> , <code>translate3D</code> , <code>rotateX/Y/Z</code> , <code>scale3D</code> y el operador de multiplicacion.
<code>draw2D.h</code>	Provee <code>Color</code> , <code>Texture2D</code> , <code>Shader</code> , <code>Shape</code> , <code>ShapeGroup</code> , carga de texturas con <code>loadTexture2D</code> , matrices globales y blending alfa/aditivo.

<code>draw3D.h</code>	Agrega <code>GenShape</code> para OBJ y geometria propia. Tambien define <code>MeshVertex</code> , <code>Sphere</code> , <code>GlowHalo</code> y <code>GlowCone</code> .
<code>lighting.h</code>	Define <code>Material</code> , <code>Light</code> , <code>LightingEffects</code> , <code>clearLights</code> , <code>addDirectionalLight</code> , <code>addPointLight</code> , <code>addSpotLight</code> y el shader Blinn-Phong con niebla.
<code>camera.h</code> y <code>CameraSystem.h</code>	Implementan <code>lookAt</code> , <code>perspective</code> , camara libre, modos fijos, FOV activo, scroll e input de teclado/mouse.
<code>objetos.h</code>	Construye la ciudad mediante modelos repetidos: carretera, veredas, edificios, arboles, bancas, postes, semaforo y luces de ventanas.
<code>Porsche.h</code> y <code>Sky.h</code>	Encapsulan el auto y el cielo. El primero agrupa piezas y ruedas; el segundo genera el domo, carga la textura del cielo y dibuja sol/halo.

Las dependencias externas son GLFW para ventana e input, GLAD para cargar funciones OpenGL, OpenGL 3.3 para el render, `stb_image` para PNG y la STL de C++ para contenedores, cadenas, archivos y operaciones matematicas.

5 Funciones clave aplicadas

Las siguientes funciones son las que conectan directamente las librerias propias con la escena final:

Funcion o clase	Uso concreto en <i>Cyberpunk Velocity</i>
<code>setProjectionMatrix</code> y <code>setViewMatrix</code>	Guardan matrices globales usadas por los shaders por defecto. Antes de dibujar se actualiza la proyeccion perspectiva y la vista activa para que todos los objetos compartan la misma camara.
<code>Shape::draw</code>	Selecciona el shader correcto, sube uniformes, enlaza el VAO y llama a <code>glDrawElements</code> o <code>glDrawArrays</code> . Si el objeto tiene normales y material, usa el shader Blinn-Phong.
<code>ShapeGroup::draw</code>	Multiplica la matriz del grupo por la matriz local de cada hijo. Se usa para agrupar piezas del Porsche y para aplicar transformaciones comunes sin reescribir cada objeto.
<code>GenShape(objPath)</code>	Carga modelos OBJ de carretera, edificios, Porsche y decoracion. Convierte el archivo a vertices intercalados con posicion, color, normal y UV.
<code>loadTexture2D</code>	Carga PNG con <code>stb_image</code> , crea el objeto <code>GL_TEXTURE_2D</code> , define wrapping/filtros y opcionalmente genera mipmaps.
<code>enableLighting(material)</code>	Activa el material de una malla para que <code>Shape::draw</code> use <code>defaultShaderLitBlinnPhong</code> . Es la union entre geometria, textura y luces.
<code>clearLights</code> y <code>addSpotLight</code>	Reinician y reconstruyen el arreglo de luces cada frame. Esto permite que las luces de postes se mantengan coherentes con la posicion animada de la ciudad.
<code>activeViewMatrix</code> y <code>activeFov</code>	Seleccionan la vista y el campo visual segun el modo de camara. Separan la logica de input de la preparacion de matrices.
<code>PorscheCar::update</code>	Calcula la rotacion acumulada de ruedas usando <code>deltaTime</code> y velocidad actual. Luego actualiza las cuatro matrices de rueda.
<code>BloqueCiudad::setLayerOffsets</code>	Aplica traslaciones en Z a capas de ciudad. Es la base del movimiento tipo carretera infinita y del parallax entre objetos cercanos y fondo.

6 Flujo de ejecucion

El flujo de `main.cpp` se resume asi:

```
Inicializar GLFW y crear ventana
Cargar GLAD y activar depth test
Cargar ciudad, Porsche y cielo
Configurar sistema de camaras

while ventana abierta:
    calcular deltaTime con glfwGetTime()
    procesar input
    actualizar animacion, ruedas y frenado
    limpiar y registrar luces
    subir proyeccion y vista
    dibujar cielo, ciudad y Porsche
    intercambiar buffers
```

El render usa `GL_DEPTH_TEST` con `GL_LESS`. Tambien se desactiva `GL_CULL_FACE`, lo que ayuda a visualizar modelos importados incluso cuando alguna cara tiene orientacion irregular. La proyeccion se actualiza cada frame con `setProjectionMatrix(perspective(...))` y la vista con `setViewMatrix(activeViewMatrix(cameras))`.

El orden del frame evita estados obsoletos. Primero se procesa el input para que la camara y la pausa respondan antes de dibujar. Luego se calcula la animacion del auto y las capas. Despues se limpian las luces con `clearLights`; esto es importante porque las luces no se acumulan indefinidamente entre frames. Finalmente se registran ambiente, niebla, luces direccionales y spotlights antes de que cualquier malla iluminada se dibuje.

El framebuffer se limpia con color y profundidad. El cielo se dibuja primero porque representa fondo; despues se dibuja la ciudad, que agrega luces de postes y emisores; finalmente se dibuja el Porsche. En el vehiculo, los cristales se dibujan despues de las piezas opacas y con escritura de profundidad desactivada para evitar que un vidrio transparente bloquee visualmente elementos posteriores.

Los estados de OpenGL se cambian de forma puntual. `GL_DEPTH_TEST` queda activo para la escena principal, mientras que el cielo lo desactiva temporalmente porque siempre debe comportarse como fondo. Para transparencias se activa `GL_BLEND`; en cristales se usa mezcla alfa tradicional y en halos/haces se usa blending aditivo. Esta separacion evita que un efecto visual como el halo del sol oscurezca la escena o que un vidrio escriba profundidad como si fuera opaco.

Estado o llamada	Motivo de uso
<code>glEnable(GL_DEPTH_TEST)</code>	Permite que carretera, edificios y auto se oculten correctamente segun profundidad.
<code>glDisable(GL_CULL_FACE)</code>	Hace visible geometria importada aunque alguna cara tenga orientacion inconsistente.
<code>glDepthMask(GL_FALSE)</code>	Evita que objetos transparentes o glow bloqueen objetos dibujados posteriormente.
<code>glBlendFunc(GL_SRC_ALPHA, GL_ONE)</code>	Suma color para halos, haces y emisores, generando apariencia luminosa.
<code>glViewport(...)</code>	Se actualiza en el callback de framebuffer para mantener la proyeccion ajustada al tamano real.

7 Transformaciones y camaras

El sistema matematico usa matrices 4x4 en formato column-major. La composicion comun es:

```
T * Rz * Ry * Rx * S
```

Esto aplica escala en espacio local, luego rotaciones y finalmente traslacion. En `objetos.h`, `makeTransform` centraliza la conversion de coordenadas exportadas desde Blender: posicion OpenGL como (X, Z, -Y), escala como (ScaleX, ScaleZ, ScaleY) y rotaciones adaptadas al sistema de ejes del proyecto.

La camara usa proyeccion perspectiva con planos 0.1 y 350.0. `CameraSystem` ofrece seis vistas: general, conductor, superior, lateral izquierda, lateral derecha y libre. Las vistas fijas usan `lookAt`; las vistas general y conductor permiten yaw/pitch limitados, y la camara libre usa WASD/QE y mouse para navegar como en un entorno de primera persona.

La funcion `lookAt` construye la base de la camara con tres vectores: `f` como direccion frontal normalizada, `s` como eje lateral por producto cruz y `u` como eje vertical corregido. Luego escribe estos ejes en una matriz column-major y calcula la traslacion con productos punto. Esta implementacion es importante porque el resto del motor asume el mismo orden de memoria que OpenGL.

En `CameraSystem`, las teclas 1-5 y L cambian el modo solo una vez por pulsacion mediante `pressedOnce`. Las camaras general y conductor no se mueven libremente: modifican offsets de yaw/pitch con limites para mantener una vista cinematografica. La camara libre, en cambio, recentra el mouse en la ventana, mide el desplazamiento contra el centro y actualiza yaw/pitch con `Camera::rotateFromMouse`. El scroll modifica el FOV entre 20 y 75 grados.

8 Carga de modelos y texturas

`GenShape` carga los OBJ leyendo posiciones `v`, coordenadas UV `vt`, normales `vn` y caras `f`. Las caras con mas de tres vertices se triangulan; si falta una normal, se genera con producto cruz. Luego la geometria se sube al GPU como atributos intercalados: posicion, color, normal y UV. En `draw2D.h`, `uploadLit` asigna estos datos a los atributos 0, 1, 2 y 3 del VAO.

Las texturas se cargan con `loadTexture2D`, que utiliza `stbi_load`. Las opciones permiten invertir la imagen verticalmente, usar formato sRGB, controlar mipmaps y elegir entre `GL_REPEAT` o `GL_CLAMP_TO_EDGE`. El Porsche toma sus mapas desde `Models/Porsche/Textures/`; la ciudad desde `Models/Road/Texture/`; y el cielo desde `Models/Sky_Synthwave_Stars.png`.

La ruta de carga de OBJ tiene dos detalles relevantes. Primero, si la ruta directa falla, `GenShape` intenta resolverla relativa al directorio de la cabecera; esto facilita ejecutar el binario desde carpetas distintas. Segundo, el parser acepta indices positivos y negativos de OBJ, lo que permite usar archivos exportados por herramientas que referencian vertices de forma relativa. El resultado final no conserva la estructura de materiales del OBJ, pero si conserva normales y UV, que son suficientes para aplicar texturas y luces definidas por codigo.

En texturas, `TextureOptions::srgb` permite subir mapas difusos como `GL_SRGB` o `GL_SRGB_ALPHA`. En el Porsche se usa esta ruta para que los colores de textura se comporten mejor bajo iluminacion. Para hojas y texturas con alfa, el proyecto combina textura RGBA con `alphaCutoff`; para la carretera y veredas se usa textura opaca con wrapping controlado.

El formato final de vertice para mallas iluminadas ocupa 12 flotantes. La posicion local define la geometria, el color por vertice permite tintes simples, la normal alimenta el calculo de luz y las UV permiten muestrear mapas difusos:

```
layout(location = 0) vec3 aPos;
layout(location = 1) vec4 aColor;
layout(location = 2) vec3 aNormal;
layout(location = 3) vec2 aTexCoord;
```

Esta organizacion permite que las primitivas generadas por codigo y los OBJ importados pasen por el mismo camino de render. Si una forma no tiene normales o material iluminado, `Shape::draw` usa un shader mas simple; si tiene normales y `lightingEnabled`, se usa el shader Blinn-Phong.

9 Iluminacion, materiales y niebla

La iluminacion se basa en Blinn-Phong. Cada material contiene ambiente, difuso, especular, emisivo, brillo, opacidad, recorte alfa y textura difusa. El shader combina ambiente global, iluminacion difusa, reflejo especular, emisividad, atenuacion por distancia, suavizado de spotlights y niebla.

En cada frame se llama a `clearLights` y luego se vuelven a registrar las luces activas. La escena usa ambiente global (0.20, 0.20, 0.22), niebla con color (0.32, 0.33, 0.44), rango 2.0-100.0 y dos luces direccionales neon: magenta y cian, ambas con intensidad 1.2.

Los postes agregan spotlights desde `LightPostObjects`: intensidad 3.2, angulo interno de 20 grados, externo de 62 grados y rango 20. Ademas, dibujan emisores visibles con un panel luminoso y `GlowCone`. `MAX_LIGHTS` vale 12; normalmente se usan 2 direccionales y 8 spotlights, por lo que la escena queda dentro del limite del shader.

Para hojas y texturas con transparencia se usa `alphaCutoff`; los fragmentos con alfa bajo se descartan. Para cristales, halos, haces de luz y semaforo se usa blending, desactivando temporalmente la escritura al depth buffer con `glDepthMask(GL_FALSE)` cuando corresponde.

El vertex shader iluminado transforma la posicion local con `uModel` y calcula la normal con `mat3(transpose(inverse(uModel)))`. Esto evita que escalas no uniformes deformen la direccion de las normales. En el fragment shader, la luz direccional usa `-direction`; las luces puntuales y spotlights calculan el vector desde el fragmento hasta la posicion de luz. La especularidad usa el vector medio $H = \text{normalize}(L + V)$, propio de Blinn-Phong.

La atenuacion de luces no direccionales depende de la distancia normalizada al rango de la luz. Para spotlights se multiplica ademas por una transicion suave entre `innerCutoff` y `outerCutoff`. La niebla mezcla el color calculado con `fogColor` segun la distancia en espacio de vista; por eso los edificios y carretera se integran al ambiente nocturno en lugar de cortar abruptamente contra el fondo.

Los materiales mas importantes no se definen por archivo MTL, sino por grupo visual. La siguiente tabla resume la intencion tecnica de cada conjunto:

Grupo	Configuracion visual
Pintura del Porsche	Material sin mapa difuso principal, con especularidad alta y <code>shininess = 96.0</code> . Busca reflejo fuerte sobre la carroceria.
Ruedas y piezas internas	Materiales texturizados con mapas PNG. Mantienen detalle visual sin modelar cada pequeno patron en geometria.
Cristales	Material con opacidad parcial y especularidad alta. Se dibuja con alpha blending y sin escribir profundidad.
Hojas de arboles	Texturas RGBA con <code>alphaCutoff</code> ; se recortan pixeles transparentes para evitar planos rectangulares visibles.
Postes, sol y ventanas	Materiales emisivos o geometria con blending aditivo para producir brillo visible en la escena nocturna.

10 Ciudad y Porsche

La ciudad esta encapsulada en `Objetos::BloqueCiudad`. El patron usado es cargar un OBJ base una vez y dibujarlo varias veces con distintas matrices mediante `RepeatedModel`. Asi se reutilizan modelos de carretera, veredas, bancas, postes, arboles, edificios, semaforo y construcciones sin duplicar toda la geometria en codigo.

`BloqueCiudad` separa dos capas: `capa_cerca`, para carretera y objetos cercanos; y `capa_media`, para edificios de fondo. `setLayerOffsets(nearZ, mediumZ)` mueve ambas capas en Z. En el loop principal, `aniBloq1` controla la capa cercana y `aniBloq2` la media; como se desplazan a velocidades distintas, se obtiene sensacion de avance y profundidad. Al superar el umbral, las capas vuelven al inicio para simular continuidad.

PorscheCar carga el vehiculo por piezas: pintura, base, carbono, cristales, motor, rejillas, interior, luces, placa, parabrisas y ruedas. Las piezas se agrupan en **opaqueGroup**, **wheelGroup** y **glassGroup**. La pintura usa un material especular con **shininess = 96.0**; las ruedas y partes internas usan mapas difusos; los cristales se dibujan al final con alpha blending para conservar transparencia.

La rotacion de las ruedas se actualiza con:

```
wheelRotation += speed * deltaTime * wheelRotationFactor;
```

Cada rueda combina traslacion, rotacion lateral, giro sobre X y una pequena compensacion del eje local para evitar que el modelo parezca tambalear.

En la ciudad, los postes tienen una doble representacion: como geometria OBJ iluminada y como fuente visual. **LightPostObjects::addLights** calcula la posicion real del foco transformando un punto local del poste a mundo y crea un spotlight apuntando hacia la carretera. **drawEmitters** dibuja el panel brillante y el **GlowCone**; este ultimo no ilumina por si mismo, pero comunica visualmente el haz de luz y aprovecha la niebla para verse integrado.

El semaforo usa el modelo OBJ para la estructura y tres circulos 2D colocados en el espacio 3D para las luces. **TrafficLigthObjects::signalState** evalua **fmod(animationTime, 10.0f)** y decide verde, amarillo o rojo. Las luces inactivas no desaparecen; se dibujan con brillo reducido para mantener la forma del semaforo.

El Porsche divide piezas por material para controlar mejor el resultado. La carroceria y las piezas opacas se dibujan primero; las ruedas se actualizan con matrices independientes; el parabrisas se coloca en **glassGroup** y se dibuja con **enableAlphaBlending**. Esta separacion evita mezclar transparencia con partes opacas y permite que la rotacion de ruedas no afecte al resto del auto.

La ciudad no crea una clase generica compleja para todos los objetos. En su lugar, cada categoria tiene una clase pequena con su ruta OBJ, su material y sus matrices. Esta decision mantiene el codigo explicito: al revisar **RoadObjects**, **LightPostObjects** o **TrafficLigthObjects** se ve inmediatamente que recurso carga, que textura usa y en que posiciones se instancia. Para un proyecto academico esto es mas entendible que una escena completamente data-driven.

Las ventanas iluminadas se generan como rectangulos pequenos colocados sobre las fachadas. **CityWindowLights** calcula posiciones a partir de especificaciones por edificio, decide que ventanas se encienden con un hash determinista y asigna colores variados. No son luces reales dentro del shader; son superficies aditivas que sugieren vida urbana sin consumir mas entradas del arreglo **MAX_LIGHTS**.

11 Cielo, animaciones y controles

Sky genera un domo semiesferico con vertices procedurales y carga **Sky_Synthwave_Stars.png**. El cielo se dibuja sin depth test y sin escribir profundidad; luego se dibuja el sol como esfera emisiva y un **GlowHalo** con blending aditivo. La niebla se desactiva temporalmente para el fondo y se restaura despues.

Las animaciones son manuales y dependen de **deltaTime**. **animationTime** acumula el tiempo global; la barra espaciadora pausa o reanuda. El semaforo sigue un ciclo de 10 segundos: verde de 0 a 5, amarillo de 5 a 7 y rojo de 7 a 10. Si el semaforo esta adelante y no esta verde, el **speedMultiplier** baja hacia cero; al liberarse, vuelve progresivamente a 1.

Controles principales: **ESC** cierra, **SPACE** pausa, 1-5 cambian entre camaras fijas, L activa camara libre, W/A/S/D y flechas ajustan vistas o movimiento, Q/E suben y bajan en camara libre, el mouse controla la orientacion y el scroll ajusta el FOV.

El cielo usa una geometria generada en **Sky::createSkyDome**. Los vertices se distribuyen por stacks y slices; las UV cubren horizontalmente el domo y ajustan verticalmente la textura para aprovechar mejor la imagen. El shader propio mezcla la textura original con una version desplazada en U cerca de la union, reduciendo la costura visual del mapa panoramico.

La animacion del avance no mueve el Porsche en Z; mueve el mundo alrededor de el. `aniBloq1` avanza la capa cercana y `aniBloq2` avanza la capa media a una fraccion de la velocidad. Esto genera la ilusion de desplazamiento continuo con el vehiculo como referencia visual fija. Cuando una capa supera el limite, se reinicia a una posicion anterior para que la carretera parezca repetirse.

El frenado combina el estado del semaforo con una distancia estimada al punto de cruce. Si la luz no esta verde y el semaforo esta delante dentro de 15 unidades, `targetMultiplier` pasa a cero; si no, vuelve a uno. La variable `speedMultiplier` interpola hacia ese objetivo con una tasa mayor al frenar que al acelerar, lo que da un comportamiento mas natural que un cambio instantaneo.

12 Trazabilidad visual y tecnica

La escena puede entenderse como una correspondencia entre elementos visibles y decisiones de implementacion. Esto ayuda a ubicar rapidamente donde se produce cada efecto:

Elemento visible	Implementacion relevante
Carretera en movimiento	<code>BloqueCiudad::setLayerOffsets</code> desplaza <code>capa_cerca</code> ; el reinicio del offset en <code>main.cpp</code> evita que el camino se termine visualmente.
Profundidad urbana	<code>capa_media</code> avanza mas lento que <code>capa_cerca</code> . Esta diferencia crea parallax sin mover la camara ni el auto.
Ambiente nocturno cyberpunk	Dos luces direccionales coloreadas, materiales emisivos, ventanas aditivas, cielo texturizado y niebla azulada forman la paleta visual.
Semaforo funcional	<code>TrafficLightObjects</code> dibuja el estado visible; <code>main.cpp</code> usa el mismo tiempo de ciclo para decidir si el vehiculo debe frenar.
Vidrios y halos	Se renderizan con blending y <code>glDepthMask(GL_FALSE)</code> para que sean transparentes o luminosos sin corromper el depth buffer.
Ruedas animadas	<code>PorscheCar::update</code> acumula rotacion proporcional a velocidad y <code>updateWheelTransforms</code> aplica la matriz individual a cada rueda.

La ventaja de esta organizacion es que cada efecto importante tiene un punto claro de control. Las luces reales estan en el sistema de iluminacion; los brillos decorativos son geometria aditiva; el movimiento de mundo esta en offsets de capa; y la interaccion se concentra en `CameraSystem` y en el loop principal. Esta separacion reduce acoplamiento y facilita explicar el proyecto desde el codigo.

13 Aspectos tecnicos

El orden de dibujo es importante: primero se limpia el frame, luego se dibuja el cielo sin depth test, despues la ciudad opaca, los elementos glow y finalmente el Porsche con cristales al final de su propio metodo. Esto reduce errores de transparencia y aprovecha el depth buffer para los objetos solidos.

El instanciado es manual: un mismo `GenShape` se dibuja varias veces cambiando su matriz de modelo. Es una solucion simple y clara para el proyecto, aunque una version mas optimizada podria usar `glDrawElementsInstanced`. Tambien podria activarse `GL_CULL_FACE` si se corrige la orientacion de todos los modelos.

Una decision importante es no usar instancing por GPU. El costo de cambiar matrices y redibujar objetos repetidos es aceptable para el tamano academico de la escena y hace que la implementacion sea mas facil de seguir. Si se buscara escalar a cientos o miles de objetos, el siguiente paso seria agrupar matrices en un buffer de instancias y usar `glDrawElementsInstanced`.

Otra decision es mantener materiales en C++. Esto evita depender de diferencias entre exportadores MTL y permite ajustar rapidamente el aspecto cyberpunk: luces neon, emisividad, cristales, halos y

niebla. La desventaja es que al agregar nuevas piezas se debe crear o asignar manualmente su material y textura.

El proyecto tambien separa lo visual de lo fisico de manera practica. Por ejemplo, un poste agrega un spotlight que afecta la iluminacion real y ademas dibuja un cono transparente que representa el haz. El semaforo usa circulos brillantes para mostrar estado, pero el frenado del auto se decide por tiempo y distancia calculada en `main.cpp`. Esta separacion simplifica el control de la escena: la parte visible comunica el estado y la logica mantiene la animacion.

14 Conclusion

Cyberpunk Velocity integra una escena 3D completa con una arquitectura entendible: `main.cpp` coordina el loop, `CameraSystem.h` maneja vistas e input, `objetos.h` construye la ciudad, `Porsche.h` encapsula el vehiculo, `Sky.h` controla el fondo y `draw2D/draw3D/lighting` proveen la base grafica. El resultado demuestra transformaciones, camaras, carga de mallas, texturizado, materiales, iluminacion, blending, niebla y animacion temporal dentro de una misma aplicacion.